

Introduction au Machine Learning en génomique

Applications à l'antibiorésistance : apprentissage supervisé et non supervisé

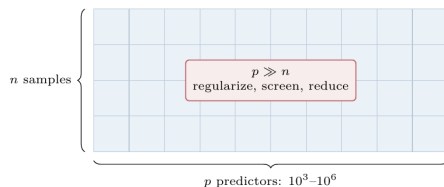
Imad EL BADISY, PhD

11 septembre 2026

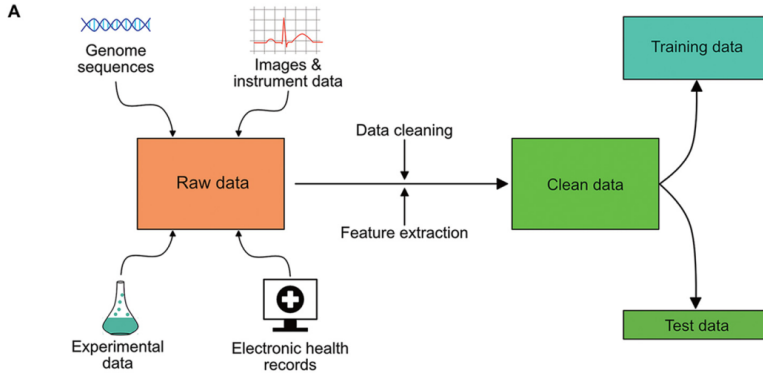
Partie 1 : Le machine learning en génomique : applications à l'antibiorésistance

Pourquoi le machine learning en génomique

- les modèles classiques supposent peu de prédicteurs, choisis à la main
- fortes corrélations (déséquilibre de liaison), interactions, effets non linéaires
- 10^3 à 10^6 variables par échantillon : positions, gènes, k -mers, sites de méthylation
- le ML apprend $\hat{f}(x) \rightarrow y$ et **évalue** sur des données non vues (test), fait de la sélection de variables à grande échelle, découvre une structure sans label

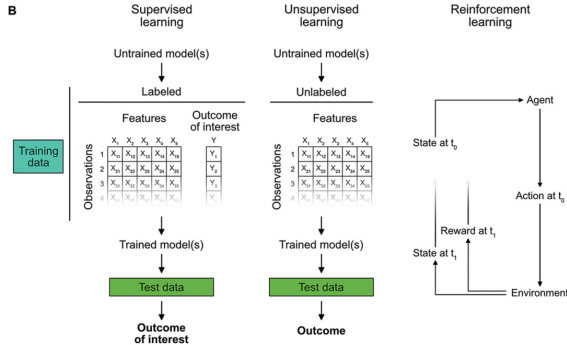


De la donnée brute au modèle entraîné



Anahtar, Yang & Kanjilal (2021), *J. Clin. Microbiol.* 59:e01260-20, Fig. 1A.

Trois types d'apprentissage



Anahtar, Yang & Kanjilal (2021), Fig. 1B. Ce cours couvre les deux types de gauche : **supervisé** (funcml) et **non supervisé** (phynotype) ; l'apprentissage par renforcement est hors périmètre.

Apprentissage supervisé, formellement

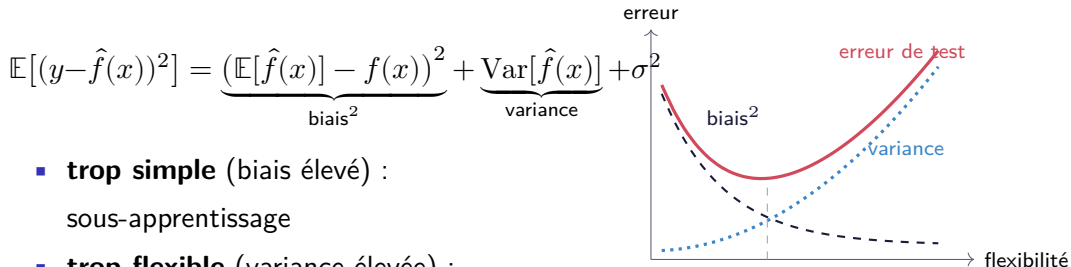
Données $(x_i, y_i)_{i=1}^n$, on cherche $\hat{f}$ tel que $\hat{f}(x) \approx y$:

$$\hat{f} = \arg \min_{f \in \mathcal{F}} \frac{1}{n} \sum_{i=1}^n L(y_i, f(x_i))$$

- y quantitatif $\rightarrow$ régression (RMSE, MAE)
- y qualitatif $\rightarrow$ classification : $\hat{p}(x) = \Pr(y = 1 \mid x)$, puis une classe via un seuil t
- le risque $R(\hat{f}) = \mathbb{E}[L(y, \hat{f}(x))]$ s'estime sur des données **non vues**, jamais sur les données d'entraînement

Le compromis biais-variance

Erreur de test = biais² + variance + bruit irréductible

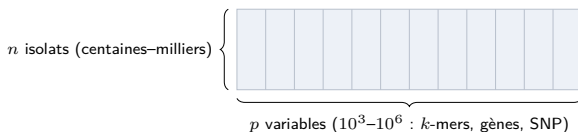


- **trop simple** (biais élevé) :
sous-apprentissage
- **trop flexible** (variance élevée) :
surapprentissage
- objectif : le creux de la courbe d'erreur
de **test**

La flexibilité utile est un **hyperparamètre** : réglé par validation croisée, jamais sur

Grande dimension ($p \gg n$) et la malédiction qui l'accompagne

Le régime typique en génomique, et particulièrement en génomique de l'AMR



- sans contrainte, une infinité de modèles ajustent parfaitement les données d'entraînement ; quand $p \rightarrow \infty$, presque tous les points deviennent équidistants, les notions de voisinage (k -NN, noyaux) se dégradent
- tester $m = 10^5$ variables à $\alpha = 0.05$ donne $\mathbb{E}[\text{faux positifs}] \approx 5\,000$, la sélection doit se faire **à l'intérieur** de la validation croisée, jamais avant
- réponses : **régularisation** (lasso/ridge/elastic net), **filtrage**, **réduction de dimension** (PCA, embeddings)

Validation croisée à k blocs

Chaque observation sert une fois à l'évaluation

bloc 1	test	train	train	train	train	→ score ₁	} moyenne ± écart-type
bloc 2	train	test	train	train	train	→ score ₂	
bloc 3	train	train	test	train	train	→ score ₃	
bloc 4	train	train	train	test	train	→ score ₄	
bloc 5	train	train	train	train	test	→ score ₅	

$$CV(\hat{f}) = \frac{1}{v} \sum_{k=1}^v L\left(y^{(k)}, \hat{f}^{(-k)}(x^{(k)})\right)$$

Stratifiée par défaut ; **groupée** quand les observations sont liées. Réglage + comparaison

~~+ estimation finale~~ : sur des rééchantillonnages **distincts** (validation croisée imbriquée).²

2. Varma & Simon (2006), *BMC Bioinformatics* 7:91.

Cinq pièges qui fragilisent le ML en génomique

- **différences de distribution** : train/test tirés de distributions différentes (lot, espèce, plateforme)
- **exemples dépendants** : isolats liés par une même lignée clonale, viole l'indépendance
- **facteurs de confusion** : la structure de population bactérienne prédit seule la résistance, sans lien causal direct
- **fuite de données** : prétraitement (standardisation, imputation, sélection) ajusté sur l'ensemble complet avant la séparation train/test
- **déséquilibre de classes** : accuracy trompeuse quand une classe est rare

REVIEWS



Navigating the pitfalls of applying machine learning in genomics

Sean Whalen^{1,6}, Jacob Schreiber^{2,6}, William S. Noble³ and Katherine S. Pollard^{1,4,5} 

Abstract | The scale of genetic, epigenomic, transcriptomic, cheminformatic and proteomic data available today, coupled with easy-to-use machine learning (ML) toolkits, has propelled the application of supervised learning in genomics research. However, the assumptions behind the statistical models and performance evaluations in ML software frequently are not met in biological systems. In this Review, we illustrate the impact of several common pitfalls encountered when applying supervised ML in genomics. We explore how the structure of genomics data can bias performance evaluations and predictions. To address the challenges associated with applying cutting-edge ML methods to genomics, we describe solutions and appropriate use cases where ML modelling shows great potential.

Nat. Rev. Genet. 23:169–181, “Navigating the pitfalls of applying machine learning in genomics.”

Deux façons de prédire la résistance à partir d'un génome

Basée sur des règles

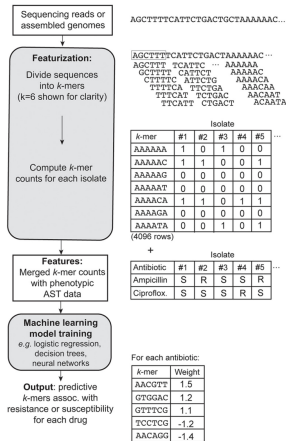
- présence/absence de gènes connus (*mecA*, *vanA*) ou de mutations (*gyrA* S83L)
- excellent pour une résistance simple, bien caractérisée ; difficile à faire évoluer

ML génotypique

- apprend les caractéristiques prédictives directement des données, sans base de gènes organisée
- passe à l'échelle sur des mécanismes non encore caractérisés, au prix d'une interprétation moins immédiate

Exemple de pipeline : featurisation par k -mers

B Example of model training



Anahtar, Yang & Kanjilal (2021), Fig. 2B, génome → décomposition en k -mers → table isolat × compte → entraînement → k -mers pondérés associés à la résistance.



Applications of Machine Learning to the Problem of Antimicrobial Resistance: an Emerging Model for Translational Research

 Melis N. Anahtar,^a  Jason H. Yang,^{b,c}  Sanjat Kanjilal^{d,e}

^aDepartment of Pathology, Massachusetts General Hospital, Boston, Massachusetts, USA

^bCenter for Emerging Pathogens, New Jersey Medical School, Rutgers University, Newark, New Jersey, USA

^cDepartment of Microbiology, Biochemistry and Molecular Genetics, New Jersey Medical School, Rutgers University, Newark, New Jersey, USA

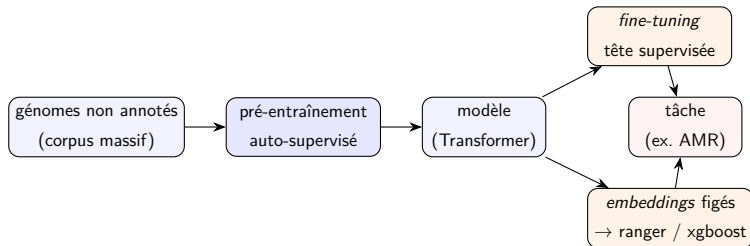
^dDepartment of Population Medicine, Harvard Medical School & Harvard Pilgrim Healthcare Institute, Boston, Massachusetts, USA

^eDivision of Infectious Diseases, Brigham & Women's Hospital, Boston, Massachusetts, USA

J. Clin. Microbiol. 59:e01260-20, “Applications of machine learning to the problem of antimicrobial resistance.”

Autre voie de featurisation : les modèles de fondation

Pré-entraînés par auto-supervision sur de larges corpus de génomes non annotés, puis réutilisés en aval :



- exemples : **DNABERT-2**, **Nucleotide Transformer**, **HyenaDNA** (séquence); **Enformer** (régulation)
- utile quand la tâche part de la séquence brute avec peu de données étiquetées; superflu quand les prédicteurs sont déjà tabulaires (notre cas ciprofloxacine), établir

Le cas *E. coli* / ciprofloxacin : ce que dit la littérature

D'après Moradigaravand *et al.* (2018), *PLoS Comput. Biol.* 14:e1006258, 1 936 génomes d'*E. coli*, 11 antibiotiques :

- le **boosting de gradient** gagne sur 11/11 antibiotiques ; la présence/absence de gènes est utilisée dans **tous** les meilleurs modèles
- pour la ciprofloxacin, la présence de gènes seule ne suffit pas, il n'existe pas de gène unique de résistance
- ajouter les SNP du génome cœur améliore seulement 2/11 antibiotiques, dont la ciprofloxacin (+2,8 %), via **gyrA** et **parC**, les régions connues de résistance aux quinolones

Une leçon directe pour la suite : la structure de population (ST131 est intrinsèquement résistant) explique une partie du signal, le piège de confusion vu plus haut, dans des données réelles.

RESEARCH ARTICLE

Prediction of antibiotic resistance in *Escherichia coli* from large-scale pan-genome data

Danesh Moradigaravand ^{1,2*}, Martin Palm ^{3,4}, Anne Farewell ^{3,4}, Ville Mustonen ^{5,6},
Jonas Warringer ^{3,4}, Leopold Parts ^{1,7*}

1 Wellcome Sanger Institute, Wellcome Genome Campus, Hinxton, Cambridgeshire, United Kingdom, **2** Center for Computational Biology, Institute of Cancer and Genomic Sciences, University of Birmingham, Birmingham, United Kingdom, **3** Department for Chemistry and Molecular Biology, University of Gothenburg, Gothenburg, Sweden, **4** Centre for Antibiotic Resistance Research at the University of Gothenburg, Gothenburg, Sweden, **5** Organismal and Evolutionary Biology Research Programme, Department of Computer Science, Institute of Biotechnology, University of Helsinki, Helsinki, Finland, **6** Helsinki Institute for Information Technology HIIT, Helsinki, Finland, **7** Department of Computer Science, University of Tartu, Tartu, Estonia

* d.moradigaravand@bham.ac.uk (DM); leopold.parts@sanger.ac.uk (LP)



PLoS Comput. Biol. 14:e1006258, "Prediction of antibiotic resistance in *Escherichia coli* from large-scale pan-genome data."

À retenir

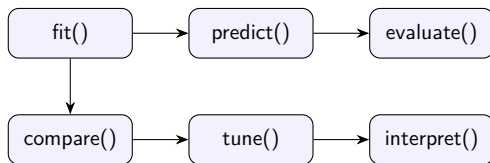
- la prédiction s'**évalue**, elle ne s'affirme pas, $p \gg n$ et les exemples dépendants sont la norme en génomique, pas l'exception
- cinq pièges reviennent sans cesse : décalage de distribution, dépendance, confusion, fuite, déséquilibre
- la présence/absence de gènes prédit bien la plupart des phénotypes de résistance chez *E. coli*; la ciprofloxaciné a nécessité un niveau SNP que les gènes seuls ne fournissaient pas
- suite : deux sessions pratiques sur **nos propres données** de résistance d'*E. coli* (data/ecoli_amr_isolate_v2.csv, 700 isolats, 8 antibiotiques)

Partie 2 : Apprentissage supervisé : comparer plusieurs algorithmes (funcml)

Le package funcml

- une interface explicite pour tout le flux : `fit()`, `predict()`, `evaluate()`, `tune()`, `compare()`, `interpret()`, `estimate()`
- 27 apprenants enregistrés : linéaires (`glm`, `glmnet`), arbres (`rpart`, `ctree`), forêts (`ranger`, `randomForest`, `cforest`), boosting (`gbm`, `xgboost`, `lightgbm`), distance/marge (`kknn`, `e1071_svm`), additifs (`gam`, `earth`), réseaux de neurones (`dmlp`, `mlp`, `nnet`), ensembles (`stacking`, `superlearner`)
- même appel `fit(formula, data, model = "...")` pour les 27, un seul mot change

Le pipeline funcml, en un schéma

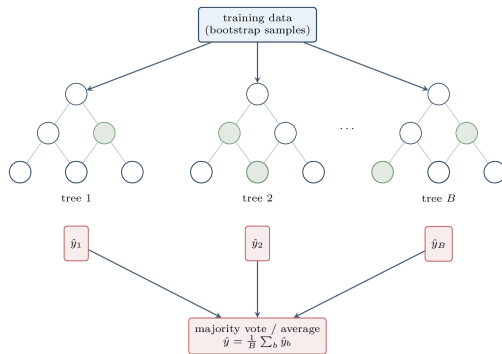


- `fit` ajuste un apprenant unique ; `compare` ajuste **plusieurs** apprenants sous le même rééchantillonnage
- `tune` règle les hyperparamètres par validation croisée ; `interpret` explique le modèle ajusté (importance, effets)

Les familles d'algorithmes en un coup d'œil

- **linéaires / pénalisés** : glm (référence interprétable), glmnet (lasso/ridge pour $p \gg n$)
- **arbres uniques** : rpart (CART, glouton), ctree (test d'indépendance formel)
- **forêts** : ranger/randomForest (bagging + sous-espace aléatoire), cforest
- **boosting** : gbm, xgboost (second ordre + régularisation), lightgbm, adaboost
- **distance / marge** : kkn (k plus proches voisins), e1071_svm (marge maximale, noyau)
- **additifs / splines** : gam, earth (MARS)
- **réseaux de neurones** : dmlp, mlp, nnet
- **ensembles** : stacking, superlearner, combinaison pondérée, réglée par validation croisée

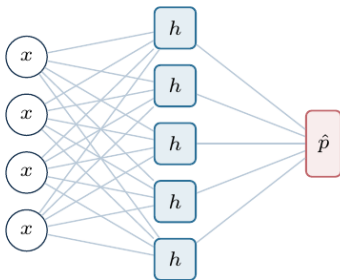
Zoom : les forêts aléatoires



each tree: random feature subset + bootstrap rows $\Rightarrow$ decorrelated errors

Chaque arbre voit un tirage bootstrap des lignes et un sous-ensemble aléatoire de variables à chaque coupe $\rightarrow$ des erreurs décorréliées entre arbres, moyennées au vote.

Zoom : les réseaux de neurones



weights + nonlinear activation

Une couche cachée h combine les prédicteurs x par des poids puis une activation non linéaire ; la sortie $\hat{p}$ est une fonction composée, apprise par descente de gradient (`dmlp`, `mlp`, `nnet`).

Nos données : résistance à la ciprofloxacine

- data/ecoli_amr_isolate_v2.csv : 700 isolats d'*E. coli*, 8 antibiotiques testés
- cible retenue : **ciprofloxacine**, seule colonne sans donnée manquante (700/700), et bien répartie (144 résistants / 539 sensibles)
- prédicteurs génomiques disponibles pour tous les isolats : genome_length, gc_content, contigs, collection_year
- collection_year comporte des valeurs manquantes (16 %) → imputation nécessaire avant modélisation

Prétraitement : imputation par `missknn`

`missknn` impute chaque valeur manquante par une moyenne pondérée par la distance de ses k plus proches voisins complets (imputation multivariée, pas juste la moyenne de la colonne) :

```
library(missknn)
imp <- missknn(model_dat[vars], k = 5, seed = 42)
model_dat[vars] <- complete(imp)
```

- `genome_length`, `gc_content`, `contigs` : aucune valeur manquante
- `collection_year` : 114 valeurs manquantes avant imputation, 0 après

`n` = 683 isolats après retrait des 17 résultats "Intermediate" (ni résistant ni sensible).

Comparer plusieurs algorithmes avec `compare()`

Même rééchantillonnage, plusieurs modèles à la fois, la bonne façon de choisir un algorithme, plutôt que d'en essayer un seul :

```
compare_obj <- compare(  
  data = model_dat,  
  formula = ciprofloxacin ~ genome_length + gc_content +  
    contigs + collection_year,  
  models = c("glm", "rpart", "ranger", "xgboost", "kkn", "e1071_svm"),  
  metrics = c("accuracy", "auc", "f1", "balanced_accuracy"),  
  resampling = cv(v = 5, seed = 42), seed = 42  
)
```

Comment choisir entre plusieurs modèles

- `compare()` renvoie, pour chaque modèle $\times$ métrique, une moyenne **et** un intervalle de confiance sur les plis de validation croisée
- on retient le modèle sur l'**intervalle**, pas seulement sur la moyenne : deux modèles dont les intervalles se chevauchent ne sont pas franchement différents
- un arbre unique (`rpart`) est en général instable d'un pli à l'autre ; forêts et boosting réduisent cette variance en moyennant plusieurs arbres

Au-delà de la discrimination

- l'AUC mesure la capacité à **classer** les cas devant les témoins, elle ne dit rien de la calibration (les probabilités prédites sont-elles fiables ?)
- `roc_curve()` visualise le compromis sensibilité/spécificité sur toute la gamme de seuils, pas seulement au seuil $t = 0,5$
- `interpret(fit, method = "permute")` : on permute chaque variable et on mesure la perte de performance, plus la chute est grande, plus la variable compte pour le modèle ajusté (Watson, 2022, *Hum. Genet.* 141:1499–1513)



Interpretable machine learning for genomics

David S. Watson¹ 

Received: 20 April 2021 / Accepted: 8 October 2021 / Published online: 20 October 2021
© The Author(s) 2021

Abstract

High-throughput technologies such as next-generation sequencing allow biologists to observe cell function with unprecedented resolution, but the resulting datasets are too large and complicated for humans to understand without the aid of advanced statistical methods. Machine learning (ML) algorithms, which are designed to automatically find patterns in data, are well suited to this task. Yet these models are often so complex as to be opaque, leaving researchers with few clues about underlying mechanisms. Interpretable machine learning (iML) is a burgeoning subdiscipline of computational statistics devoted to making the predictions of ML models more intelligible to end users. This article is a gentle and critical introduction to iML, with an emphasis on genomic applications. I define relevant concepts, motivate leading methodologies, and provide a simple typology of existing approaches. I survey recent examples of iML in genomics, demonstrating how such techniques are increasingly integrated into research workflows. I argue that iML solutions are required to realize the promise of precision medicine. However, several open challenges remain. I examine the limitations of current state-of-the-art tools and propose a number of directions for future research. While the horizon for iML in genomics is wide and bright, continued progress requires close collaboration across disciplines.

À retenir

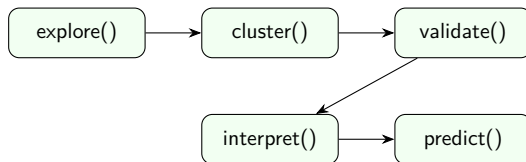
- fit → predict → evaluate → compare → tune → interpret : les mêmes verbes quel que soit l'apprenant choisi parmi les 27 enregistrés
- comparer **plusieurs** algorithmes sous le **même** rééchantillonnage est la bonne pratique, jamais un seul modèle sans concurrent
- on choisit un modèle sur l'intervalle de confiance de la métrique retenue, pas sur la seule moyenne
- la discrimination (AUC) ne suffit pas : calibration, courbe ROC et importance des variables complètent le tableau

Partie 3 : Apprentissage non supervisé : comparer plusieurs algorithmes de clustering (phynotype)

Le package phynotype

- workflow explicite : `explore()` (projection), `cluster()` (ajustement), `validate()` (indices internes/externes), `interpret()`, `predict()`
- méthodes enregistrées : "kmeans", "pam", "hclust", "agnes", "dbscan", "gmm", "kproto", "kmm"
- même logique que `funcml` : un seul mot change la méthode, tout le reste de l'appel est identique

Le pipeline phynotype, en un schéma



- explore projette les données (ACP) pour repérer une structure visuelle avant de fixer une méthode
- cluster ajuste une méthode pour un k donné; validate compare plusieurs méthodes $\times$ plusieurs k sur les mêmes indices

Panorama des méthodes de clustering

- **k-means** : minimise la somme des carrés intra-cluster $\sum_j \sum_{x_i \in C_j} \|x_i - \mu_j\|^2$, rapide, suppose des clusters sphériques
- **PAM** (Partitioning Around Medoids) : comme *k*-means mais centré sur des observations réelles, robuste aux valeurs aberrantes
- **hclust** / **agnes** : clustering hiérarchique, agglomératif, pas besoin de fixer *k* à l'avance, mais sensible aux petits groupes atypiques
- **DBSCAN** : basé sur la densité, trouve le nombre de clusters automatiquement, isole le bruit
- **GMM** : mélange de gaussiennes, affectation probabiliste (soft clustering)
- **kproto** / **KMM** : pour données mixtes (numériques + catégorielles)

Indices de validation interne

$$\text{Silhouette : } s(i) = \frac{b(i) - a(i)}{\max\{a(i), b(i)\}} \in [-1, 1] \quad \text{Calinski-Harabasz : } CH = \frac{BSS/(k-1)}{WSS/(n-k)}$$

$$\text{Davies-Bouldin : } DB = \frac{1}{k} \sum_{j=1}^k \max_{l \neq j} \frac{s_j + s_l}{d(\mu_j, \mu_l)}$$

- silhouette et CH : **plus haut** = meilleur ; Davies-Bouldin : **plus bas** = meilleur
- ces trois indices mesurent la séparation/compacité, **sans** connaître de vraie étiquette

Comparer plusieurs méthodes $\times$ plusieurs valeurs de k

`validate()` ajuste chaque méthode enregistrée pour une grille de k , sur les **mêmes** variables standardisées, et renvoie les trois indices pour chaque configuration :

```
validate(  
  data = model_dat,  
  vars = c("genome_length", "gc_content", "contigs", "collection_year"),  
  methods = c("kmeans", "pam", "hclust", "gmm"),  
  k = 2:5  
)
```

- une seule table à lire pour comparer méthodes et k , plutôt qu'un ajustement à la fois

Piège : un indice élevé peut cacher un clustering dégénéré

- un clustering hiérarchique peut afficher la **meilleure silhouette de toute la grille** en isolant seulement 2-3 observations atypiques dans un “cluster” à part
- un indice interne élevé peut donc signaler des points aberrants isolés, pas une vraie structure dans les données
- règle générale : **exclude** toute configuration où un cluster contient moins d'un seuil raisonnable des observations (p. ex. 5 %) avant de choisir sur la silhouette
- même logique que la comparaison d'algorithmes supervisés : ne jamais choisir sur un seul chiffre sans vérifier ce qu'il mesure réellement

Choisir la configuration finale

- parmi les configurations à tailles de cluster équilibrées, on retient celle avec le meilleur compromis silhouette / Calinski-Harabasz / Davies-Bouldin
- `plot_clusters()` : projette les clusters retenus en 2D (ACP) pour une inspection visuelle
- `plot_silhouette()` : une barre par observation, une silhouette négative signale une observation mal classée par rapport à son cluster
- `interpret()` : caractérise chaque cluster par ses variables les plus discriminantes

À retenir

- comparer **plusieurs méthodes et plusieurs k** sous les mêmes indices de validation, exactement comme `compare()` le fait pour l'apprentissage supervisé
- ne jamais retenir aveuglément le score le plus élevé : vérifier les tailles de cluster avant de conclure, un score parfait peut n'être que 2-3 valeurs aberrantes
- `plot_clusters()` et `plot_silhouette()` complètent les indices numériques par une inspection visuelle avant de conclure
- prochaine étape naturelle : croiser les clusters retenus avec le statut de résistance à la ciprofloxacine pour voir si la structure non supervisée recoupe le phénotype

Projet 1 : Le machine learning en génomique : applications à l'antibiorésistance